Marketplace Auth Capture — Proposal

Date: 2026-07-10 **Scope:** Landmodo and Land.com only (the two posting platforms enabled in `config/posting-platforms.json`) **Constraints (decided):** single user for now; the user always logs in themselves; the app never sees or stores marketplace passwords.

> **This markdown file is the source of truth.** Regenerate the shareable PDF (`auth-capture-proposal.pdf`) after any edit with:
> `cd workers/posting && npx tsx render-pdf.ts`

This proposal compares app-friendly alternatives to the current CLI-based login capture, verifies the claims in the seeded desk research (`research/auth-capture-spike/initial-research.md`), and ends with a drafted implementation PRD for the recommended approach. Every price or capability claim carries a source URL and fetch date, because the web changes.

---

1. Current State

How login capture works today

The ad poster (`workers/run-poster.sh` → `workers/posting/post.ts`) drives a real browser with Playwright and authenticates using a saved **storageState** JSON — cookies plus localStorage — one file per platform at `workers/posting/auth/<platform>.json`. Capturing that file is a manual, terminal-only ritual implemented in `workers/posting/capture-login.ts`:

1. The operator opens a terminal **on the worker machine** and runs

`npm run capture-login -- <platform>` from `workers/posting/`.

1. The script loads `config/posting-platforms.json`, validates that the platform

key exists and is `enabled`, and launches a **headed** Chromium (`chromium.launch({ headless: false })`) pointed at the platform's `login_url`.

1. The operator logs in by hand inside that browser window — the script never

touches credentials.

1. Back in the terminal, the operator presses Enter

(`waitForEnter(...)` over stdin); the script then serializes the browser context via `context.storageState({ path: authPath })`, writes it to `workers/posting/auth/<platform>.json`, and `chmod 600`s it.

The dashboard side is `dashboard/components/PostingAuthPanel.tsx`, mounted on the Settings page. It polls `GET /api/posting-auth` every 30 seconds and, per enabled platform, renders exactly one of two states: "Session saved (date)" (the auth file exists, with its mtime) or "No session — run `npm run capture-

`login -- <platform>`". That is the entire in-app surface: the panel is **display-only**. It cannot start a capture, refresh a session, or even tell whether a saved session is still valid — only that a file exists.

Pain points

1. **A terminal is required.** Capture is a CLI ritual (`npm run capture-login`),

not an app action. The dashboard — the product's entire operating surface for everything else — can only print the command and hope.

1. **It must run on the worker machine.** The headed Chromium and the stdin

Enter-prompt both live on the box that runs the poster. The dashboard is reachable remotely via the Cloudflare tunnel, but session capture is not: away from the machine, an expired session is unfixable.

1. **The dashboard is read-only status.** `PostingAuthPanel.tsx` shows file

existence and mtime, nothing more. There is no "Connect" or "Re-connect" button, no validity check, no in-app remediation path.

1. **Sessions expire silently.** Nothing validates a saved storageState after

capture. The first signal that a session has rotted is a **failed posting** (the poster fails fast with "run capture-login" when auth is missing or expired) — discovered after approval, when the operator expected the listing to go live.

1. **Not usable by non-technical users.** The flow assumes comfort with npm,

terminals, and the repo layout. A future assistant or second user posting ads could not self-serve a login refresh; this caps the system at one technical operator.

1. **Two-context juggling.** Even for the technical operator, the flow spans a

browser window *and* a terminal prompt that must be Enter-ed at the right moment — forget the terminal step and the login is lost.

What works and is worth preserving: no password custody (the operator types credentials directly into the real site), file permissions locked to `600`, and auth files gitignored and excluded from the file-browsing API as secrets.

---

2. Land.com — LandFeed XML API

Verdict: sanctioned, browser-free path for Land.com — recommended, pending one eligibility check with Land.com support (see open questions). The seeded research's headline claim holds up: Land.com publishes an official **LandFeed XML API** that adds, updates, and deletes listings via a single authenticated HTTPS POST, with no cookies, no browser session, and nothing to expire. If our account qualifies, this deletes the Land.com half of the auth-capture problem at its root.

How the doc fetch went (US-002 evidence)

URL	Fetched	Status	Note
`https://www.landsofamerica.com/LandFeed/Docs/`	2026-07-10	**HTTP 500**	Still errors live, exactly as the seeded research reported.
`http://web.archive.org/web/20240423062829/https://www.landsofamerica.com/LandFeed/Docs/`	2026-07-10	**HTTP 200**	Full spec recovered here — **LandFeed API Specification, Version 2.1 (September 21, 2022)**. All details below come from this snapshot.
`https://www.land.com/LandFeed/Docs/`	2026-07-10	**HTTP 200**	The `land.com` host (vs. `landsofamerica.com`) serves the same docs page live; the 500 is host-specific, so the spec is not gone, just flaky on one hostname.
`https://www.land.com/LandFeed/schemas/LandFeedSchema1.0.xsd`	2026-07-10	**HTTP 200**	The XSD the feed is validated against is live and fetchable — real, current schema.
`https://www.land.com/LandFeed/states/`	2026-07-10	**HTTP 200**	The canonical county/city name list (names must match exactly) is live.
`https://www.land.com/LandFeed/`	2026-07-10	**HTTP 200**	The POST target endpoint responds.

So the earlier "couldn't verify firsthand" caveat is now resolved: the full v2.1 spec, its XSD, and the supporting reference lists were all read directly.

What the feed does

Customers "automatically add, update, and delete land listings by securely uploading XML data to Land.com via HTTPS." One POST carries the operator's **entire** active inventory each time; Land.com diffs it against what it already has by the customer's own unique listing ID:

- **INSERT** — the listing ID is not yet in Land.com's system.
- **UPDATE** — the listing ID already exists; its data is overwritten.
- **DELETE** — a listing ID previously sent is **absent** from the new feed.

Gotcha: this makes the feed authoritative — you must send **all** listings and **all** photos every time, or anything you omit is deleted. A `mode` of `test` runs full validation without touching live listings or images.

Processing runs 365 days/year; successfully posted data is live "within minutes."

Auth model

Credentials travel **inside the XML body** (not headers), in the `<channel>` element — there is no OAuth, no cookie, no session:

- `channel.loa_account_id` (integer) — the Land.com account ID (parent account for

a corporate account). Retrievable from the Land.com Admin area.

- `channel.loa_account_email` — the email on the parent account.
- `channel.loa_shared_key` — a **secret shared key issued by Land.com technical

staff** when LandFeed is enabled for the account. This is the credential we do not yet have.

- `channel.loa_account_password` — present in the schema but **deprecated**; not

the auth path.

Transport requirements: HTTPS POST to `https://www.land.com/LandFeed/`, **TLS 1.2 or higher**, `Content-Type: text/xml`, a valid `User-Agent` header, and a correct `Content-Length`. A prerequisite stated up front: an **active Land.com Corporate Account**, with named Primary and Alternate technical contacts (this is the eligibility question below — our account may be a plain advertiser, not corporate).

For our stack this is a good fit: the shared key is a secret we'd store the same way `workers/posting/auth/*.json` are handled today (gitignored, `600`, never logged) — but unlike a browser session it never expires and never needs recapturing.

Schema (key fields)

Data format is Google Base XML (an RSS 2.0 document with a `<channel>` and repeated `<item>` elements), validated against `https://www.land.com/LandFeed/schemas/LandFeedSchema1.0.xsd`. Fields most relevant to how we generate ads today (`config/ad-platforms.json` + Ad Builder metadata):

Field	XML element	Req?	Notes
Customer listing ID	`item.id`	yes	Our own unique key; drives INSERT/UPDATE/DELETE. `task.id` is a natural fit.
Description	`item.description`	yes	CDATA. **No HTML, no URLs, no email addresses** — those listings are rejected outright.
Custom listing title	`item.listing_title`	no	Up to 775 chars — the headline surface.
Listing status	`item.listing_status`	yes	`Available` / `Contract Pending` / `Sold`.
County	`item.county`	yes	Must match Land.com's county names exactly (list at `/LandFeed/states/`).
State	`item.state`	yes	Full name or 2-char code.
Closest city	`item.closest_city`	yes	
Lot size	`item.lot_size`	yes	Acres, ≤2 decimals; residential/commercial must be ≥1 acre.
Price	`item.price`	yes	Integer, no non-numeric characters.
Property types	`item` attribute `propertytypes`	—	Bitwise sum (Recreational Land = 4, Undeveloped Land = 32, Hunting Land = 128, etc.), max 3 types; supersedes the legacy `property_type` tag.
Main photo	`item.image_link`	no	One image URL (Land.com pulls it).
Photo tour	`item.loa_photo_tour_images.image_link`	no	0–200 image URLs.
Lead routing email	`item.lead_routing_email`	no	Extra address to notify on a lead.
Sales comps	`item.salesdata.*`	no	Only when `listing_status = Sold`; ties into our knowledge-base sold-ad loop.

Photos are referenced by **URL** — Land.com GETs them from links in the XML (JPEG/ GIF/PNG/BMP, ≤2 MB each, min ~300px wide). This differs from the current poster, which uploads local files from `outputs/<taskId>/photos/`; a feed integration would need those photos reachable at a public URL.

Posting latency

- **Listings:** live in **5 minutes to ~1 hour** (per the spec's FAQ).

- **Images:** processed by a separate pass, typically **10 minutes to 6 hours**

depending on count.

- Feed and image processing each email a success/warning/error report to the

account's technical contacts; per-listing errors don't halt the rest of the feed. Results are verified by logging into the Property Control Center (`propertycontrolcenter.com`) — the same admin surface the current Playwright poster drives.

Cost

$0 beyond the Land.com listing plan the operator already pays for. LandFeed is a feature of an account, not a metered API — the only stated prerequisite is an active Corporate Account and issuance of the shared key. No per-post fee, no vendor, no infrastructure.

Open questions for Land.com support

1. **Eligibility:** Is LandFeed available on our current account tier, or does it

require a **Corporate Account** specifically? The spec lists "an active Land.com Corporate Account" as a prerequisite — the single biggest unknown, since we may be a standard advertiser.

1. **Credential issuance:** How is the `loa_shared_key` issued, to whom, and how is

it rotated/revoked if leaked? (The spec says "Land.com staff will create a new unique key.")

1. **Account IDs:** Confirm our `loa_account_id` (and any child account IDs) from

the Admin area, and whether a corporate parent must be created first.

1. **Cost:** Any fee or plan upgrade tied to enabling LandFeed?
2. **Photo hosting:** Since photos are ingested by URL, is there any Land.com-hosted

upload path, or must we serve our `outputs/<taskId>/photos/` images at a public HTTPS URL ourselves?

1. **Schema currency:** Is `LandFeedSchema1.0.xsd` (v2.1, Sept 2022) still the

current schema, and is the `landsofamerica.com/LandFeed/Docs/` 500 a known issue?

Draft email to Land.com support (ready to send)

> **To:** support@land.com (cc: sales@land.com)
> **Subject:** LandFeed XML API access for my advertiser account
>
> Hello,
>
> I advertise land listings on Land.com and would like to enable the **LandFeed
> XML API** so I can add, update, and delete my listings programmatically instead
> of entering them by hand in the Property Control Center.

> Could you help me confirm a few things?
>
> 1. Is LandFeed available on my current account, or do I need a Corporate Account to use it? If I need one, what's involved in setting that up?
> 2. How do I get my **shared key** (`loa_shared_key`) and confirm my **account ID** (`loa_account_id`)?
> 3. Is there any cost or plan change associated with enabling LandFeed?
> 4. Is `LandFeedSchema1.0.xsd` (spec version 2.1, September 2022) still the current schema? The docs page at `landsofamerica.com/LandFeed/Docs/` currently returns an HTTP 500 error, though `land.com/LandFeed/Docs/` loads.
>
> My account email is daniel@ownaloha.land. Happy to provide my account ID or set up the required technical contacts.
>
> Thank you,
> Dan

Fallback if feed access is denied

If LandFeed turns out to be gated to broker/corporate tiers we can't reach, the fallback is to **keep the current Playwright poster for Land.com** and capture its login session using whatever approach we choose for Landmodo (the hosted live-view browser recommended in §4). Nothing about Land.com's posting *mechanics* changes in that case — only the login-capture UX improves alongside Landmodo's. This keeps Land.com functional regardless of the eligibility answer; the LandFeed API is a strict upgrade we adopt only if the account qualifies.

Sources (all fetched 2026-07-10): LandFeed API Specification v2.1 via Wayback `web.archive.org/web/20240423062829/https://www.landsofamerica.com/LandFeed/Docs/`; live schema `https://www.land.com/LandFeed/schemas/LandFeedSchema1.0.xsd` (HTTP 200); live endpoint `https://www.land.com/LandFeed/` (HTTP 200); live docs mirror `https://www.land.com/LandFeed/Docs/` (HTTP 200); `landsofamerica.com/LandFeed/Docs/` (HTTP 500).

---

3. Landmodo — login-flow recon

Verdict: Landmodo needs a browser session (no API), and its login is automation-friendly — a plain email/password form that loads normally for a headless datacenter browser, guarded only by an invisible Google reCAPTCHA with no visible challenge and no MFA. This grounds the seeded research's guess ("no evidence of aggressive bot protection or MFA") in first-hand evidence and makes the hosted live-view approach in §4 a clean fit: a human logging in inside the page satisfies reCAPTCHA naturally, and the resulting session is what we persist.

How the recon was run (US-003 evidence)

A one-off headless Playwright script, `workers/posting/recon-landmodo.ts`, drove the **same Chromium install the poster uses** — default fingerprint, headless, datacenter IP, no stealth. It loaded the login page, saved a full-page screenshot, and dumped bot-protection markers and form fields. **No credentials were entered and no form was submitted — recon only.** Re-runnable with `cd workers/posting && npx tsx recon-landmodo.ts`.

Screenshot artifact: `research/auth-capture-spike/artifacts/landmodo-login.png` (fetched 2026-07-10) — shows a clean "Log Into Your Account" card with Email Address + Password fields and a "Login Now" button, and **no visible reCAPTCHA checkbox or challenge** on the login tab.

Page	Fetched	Status	Note
`https://www.landmodo.com/login`	2026-07-10	**HTTP 200**	Loads and renders fully for a headless datacenter browser — no Cloudflare interstitial, no bot wall, no block.
`https://www.landmodo.com/seller-support`	2026-07-10	**HTTP 200**	Seller FAQ; describes manual listing creation only (see below).
`https://www.landmodo.com/how-it-works`	2026-07-10	**HTTP 404**	No such page (guessed path).
`https://www.landmodo.com/list-your-company`	2026-07-10	**HTTP 404**	No such page (guessed path).

Page load for a headless datacenter-fingerprint browser

The login page returned **HTTP 200** and rendered completely — title "Login Now - Landmodo", full form visible in the screenshot — under a plain headless Chromium with a datacenter IP and no fingerprint spoofing. There was no Cloudflare challenge, no "verify you are human" interstitial, and no redirect to a bot wall. In other words, Landmodo does **not** block automated browsers at page load; the only bot defense sits on the form submission (reCAPTCHA), which we never trip because recon stops before submit.

Bot-protection / CAPTCHA / MFA markers

The recon scanned the rendered HTML for the known bot-wall / CAPTCHA / MFA vendor signatures (reCAPTCHA, hCaptcha, Cloudflare Turnstile/challenge, DataDome, PerimeterX, Arkose/FunCaptcha). Result:

- **Google reCAPTCHA — present.** The page loads Google's reCAPTCHA

(`www.google.com/recaptcha`, `g-recaptcha`), carries a `g-recaptcha` container, and the login form (`member_login_190`) posts a **hidden `recaptcha` input** the script populates. Crucially, the screenshot shows **no visible checkbox or image challenge** on the login tab — the reCAPTCHA is scored invisibly in the background, so a human logging in normally sees nothing to solve.

- **Nothing else.** No hCaptcha, no Cloudflare Turnstile or challenge platform, no

DataDome, no PerimeterX, no Arkose/FunCaptcha.

- **No MFA / OTP.** No two-factor, OTP, verification-code, or authenticator

markers anywhere on the login page — login is single-factor email + password.

Implication for our options: a **credential-vault / headless scripted login (§5d) is the riskiest** here — a datacenter-IP scripted login is exactly what an invisible reCAPTCHA scores down, and it can start silently failing at any time. A **human-driven login inside a hosted live-view browser (§4) sidesteps this entirely** — the person passes reCAPTCHA the way any real user does, and we persist the session that results.

Login form fields observed

The `member_login_190` login form exposes:

Field	type	name	Notes
Email Address	`email`	`email`	required; placeholder `name@yoursite.com`
Password	`password`	`pass`	required; placeholder `Enter Password`
reCAPTCHA token	`hidden`	`recaptcha`	populated by the reCAPTCHA script on submit
(submit)	`submit`	—	the "Login Now" button

plus hidden form-plumbing fields (`sized` , `form` , `formname` , `dowiz` , `save` , `url_origin_pars` , `action`). A separate "Register New Account" tab (`signup_free`) sits on the same page with its own email/confirm/password/consent fields — not relevant to session capture. The form is a conventional server-rendered POST — no SSO, no email-magic-link, no passkey.

No official posting API or bulk import (confirmed from landmodo.com)

The seeded research's "no API found" holds up against the site's own pages. The seller FAQ at `landmodo.com/seller-support` (HTTP 200) answers **"How do I post a Property?"** with a purely manual, dashboard-driven process — *"After you login, you are taken to the dashboard … on the left side look for Properties and click that link, this is where you will manage your listings"* — and its plan answer confirms per-listing manual entry (Starter/free plan = 3 listings, then $19.95 tiers). Nowhere in the FAQ's visible text do the words **api, bulk, csv, import, feed, xml, integration,** or **automation** appear; the only HTML hits for those strings were in generic page chrome (scripts/meta), and they showed up identically on Landmodo's 404 pages, confirming they are not feature references. There is no documented programmatic or bulk-upload path.

Action (operator task, not a Ralph story): email Landmodo support to ask whether any bulk-import or feed option exists for sellers — a small site may do something ad hoc — but the proposal plans as if the answer is no, i.e. Landmodo requires browser automation with a persisted login session.

Sources (all fetched 2026-07-10): live recon of `https://www.landmodo.com/login` (HTTP 200) via `workers/posting/recon-landmodo.ts` , screenshot at `research/auth-capture-`

`spike/artifacts/landmodo-login.png` ; seller FAQ `https://www.landmodo.com/seller-support` (HTTP 200).

---

4. Hosted Live-View Browsers

Verdict: this is the purpose-built answer for Landmodo — the operator logs in to the real site inside an iframe in our own dashboard, the session persists in a vendor-side profile, and the poster reuses it. All four vendors ship the exact "log in inside the app, persist for automation" primitive. Browserbase is the best-value battle-tested pick at $20/mo for the 20-user case; every vendor is effectively $0 at 1 user. One material change since the seeded research: **Steel.dev's cheap ~$29/mo recurring tier is gone** — its paid plan now starts at $250/mo (details below), which reshuffles the value ranking.

The flow is identical across vendors: the app creates a remote browser session bound to a **persistent profile/context**, gets a **live-view URL**, embeds it in an `<iframe>` ; the operator logs in to Landmodo *inside the dashboard* (passing Landmodo's invisible reCAPTCHA naturally, per §3); the session ends; cookies and localStorage persist in the profile; the poster later either connects to that profile over CDP or exports its cookies into the existing `workers/posting/auth/<platform>.json` storageState files.

Per-vendor findings (verified 2026-07-10)

Browserbase

	Detail
Pricing (`browserbase.com/pricing` , fetched 2026-07-10)	**Free** $0/mo: 1 browser-hour, 3 concurrent, **15-min session cap**, 7-day retention, no CAPTCHA solving. **Developer** $20/mo: 100 browser-hours then **$0.12/hr**, 25 concurrent, 1 GB proxies then $12/GB, **auto CAPTCHA solving included**. **Startup** $99/mo: 500 hrs then $0.10/hr, 100 concurrent, 5 GB proxies then $10/GB, 30-day retention. **Scale**: custom (250+ concurrent, HIPAA/BAA/DPA).
Live-view embed (`docs.browserbase.com/features/session-live-view` , fetched 2026-07-10)	`bb.sessions.debug(sessionId)` returns `debuggerFullscreenUrl` (no chrome) and `debuggerUrl` (with borders). Embed in an `<iframe>` with `sandbox="allow-same-origin allow-scripts"` and `allow="clipboard-read; clipboard-write"` ; add `style="pointer-events:none"` for read-only. `&navbar=false` maximizes the viewport. A `window message` event `browserbase-disconnected` signals session end.
Persistence (`docs.browserbase.com/features/contexts` , fetched 2026-07-10)	**Contexts** store the entire Chromium user-data dir (cookies, localStorage, IndexedDB, Session Storage, Service Workers, preferences). Create a Context → start a session with `context: { id, persist: true }` → log in → close → reuse the same Context ID to be auto-signed-in.
State back to our workers	Poster runs against the vendor browser via `connectOverCDP` reusing the Context ID; **or** pull cookies out with CDP `Network.getAllCookies` and write the existing `auth/<platform>.json` . A first-party "export storageState" endpoint is **not documented** — the generic CDP path is the reliable one.

Steel.dev

	Detail
Pricing (`steel.dev/pricing`, fetched 2026-07-10)	**CHANGED since seeded research. Launch** (free): $0/mo + usage, **$30 one-time credits**. **Scale** (popular): **$250/mo** + usage, $100 credits/mo, SSO, HIPAA-ready BAA. **Enterprise**: custom, 1,000+ concurrent. Built-in CAPTCHA solving + proxies included in credits; up to 24 h sessions; sub-1 s start. The page no longer publishes a per-hour rate or the old ~$29/mo tier; historical rate was ~$0.10/br-hr. **Self-hostable** (open-source `steel-browser`) — the escape from vendor pricing entirely.
Live-view embed (`docs.steel.dev/overview/sessions-api/human-in-the-loop`, fetched 2026-07-10)	`<iframe src="${debugUrl}?interactive=true&showControls=true">`. `interactive=true` enables clicks/scroll/typing; `showControls=true` shows a URL/back/forward bar. Actions in the interactive session mutate the real session state. Login-completion detection is **not** a built-in event — poll session state (a cookie/URL check) externally.
Persistence (`steel.dev/blog/profiles`, fetched 2026-07-10)	**Profiles** persist cookies, extensions, credentials, localStorage, auth tokens, and fingerprints. `sessions.create({ profileId })` resumes; `persistProfile: true` writes new state back (default false = read-only).
State back to our workers	`connectOverCDP` using the session ID gives full programmatic access to the authenticated browser; or export cookies via CDP as above. Same two options as Browserbase.

Anchor Browser

	Detail
Pricing (`docs.anchorbrowser.io/pricing`, fetched 2026-07-10)	Pure usage: **$0.05/browser-hour** (billed to the minute), **$0.01/session** created, **$8/GB** proxy, $0.01/AI step. **Free** plan: **$5 credits/month**. **Starter** $50/mo, **Growth** $2,000/mo, Enterprise custom; paid-tier overage +$1.00/credit.
Live-view embed (`docs.anchorbrowser.io/advanced/browser-live-view`, fetched 2026-07-10)	`live_view_url` returned at session creation; embed in `<iframe>` with `sandbox="allow-same-origin allow-scripts"` `allow="clipboard-read; clipboard-write"`; `pointer-events:none` for read-only. `one_time_url: true` (headful only) permanently invalidates the URL after the first viewer connects and disconnects — a strong fit for a single-use "Connect" hand-off.
Persistence	Profiles / identity ("OmniConnect") per the seeded research; the live-view and pricing pages were re-verified firsthand, the profiles mechanism was not independently re-fetched this iteration (flagged).
State back to our workers	CDP connect / cookie export, same as the others (Anchor exposes a standard CDP endpoint).

Hyperbrowser

	Detail
Pricing (`hyperbrowser.ai/docs/pricing`, via search, fetched 2026-07-10 — the marketing pricing page is JS-rendered and returned no data)	Credit model: **1 credit = $0.001**, **1 browser-hour = 100 credits = $0.10**. **Free**: **1,000 credits (10 br-hr) + 1 concurrent**, no card. **Startup** $30/mo (25 concurrent). **Scale** (100 concurrent). Purchased credits expire after 12 months; plan credits refresh on renewal.
Live-view embed (`hyperbrowser.ai/docs/sessions/live-view`, fetched 2026-07-10)	`<iframe src="https://app.hyperbrowser.ai/live?token=<TOKEN>">`. The `liveUrl` token **expires after 12 h** — re-GET the session for a fresh `liveUrl`. `viewOnlyLiveView: true` at session creation makes it read-only. Docs warn the URL grants control — treat it as a secret.
Persistence	Profiles (created via API/dashboard; fresh vs. resumed) per the seeded research; the live-view + pricing were re-verified, the profiles doc path 404'd this iteration (flagged).
State back to our workers	CDP connect / cookie export, same as the others.

Cost projections for our usage

Assumptions (from the PRD): a login capture ≈ **2 min** of browser time; a posting run ≈ **3 min**. Sessions persist, so captures are occasional (session-rot re-auth, roughly 1–2/user/month). If Land.com moves to the LandFeed API (§2), only **Landmodo** posts through a hosted browser; if LandFeed is denied, double the posting minutes for the both-platforms case.

- **1 user, ~10 posts/mo:** 10 posts × 3 min + ~2 captures × 2 min ≈ **0.6 browser-hours/mo**.
- **20 users, ~30 posts/mo each:** 600 posts × 3 min + ~30 captures × 2 min ≈ **31 browser-hours/mo** (≈ 630 sessions).

Vendor	1 user (~0.6 br-hr/mo)	20 users (~31 br-hr/mo)
Browserbase	**$0** (Free: 1 br-hr/mo incl.). Caveat: 15-min session cap could truncate a slow login.	**$20/mo** Developer (100 br-hr + CAPTCHA solving + 25 concurrent, all within cap). Best value + most battle-tested.
Steel.dev	**$0** (Launch one-time $30 credits ≈ 300 br-hr — depletes over time).	~**$3/mo usage** on Launch after the one-time credits, **but no cheap recurring tier** — next formal plan is **$250/mo**. Or **self-host** for $0 vendor cost.
Anchor	**$0** (Free $5/mo credits ≈ 100 br-hr at $0.05/hr; session fees trivial).	~**$8/mo pay-as-you-go** (31 br-hr × $0.05 = $1.55 + ~630 sessions × $0.01 = $6.30), partly offset by $5/mo free credits; Starter $50/mo if a plan is required for support/concurrency.
Hyperbrowser	**$0** (Free 1,000 credits = 10 br-hr/mo).	~**$30/mo** Startup (usage ≈ 3,100 credits = $3.10, but 25-concurrent needs the Startup plan).

At **1 user, every vendor is free.** At **20 users, Browserbase Developer ($20/mo, CAPTCHA solving bundled) is the cheapest turnkey recurring option**; Anchor is cheaper on raw pay-as-you-go but adds per-session accounting and (for support/ concurrency) a $50/mo floor; Hyperbrowser is $30/mo; Steel is now either self-host or $250/mo. These are near-noise costs at this scale regardless.

Integration sketch for our stack

Today `dashboard/components/PostingAuthPanel.tsx` is display-only (§1). The change turns each platform row into an actionable **"Connect Landmodo"** control:

1. **"Connect" button** in `PostingAuthPanel.tsx` calls a new dashboard route,

e.g. `POST /api/posting-auth/connect` with `{ platform }`.

1. That route uses the chosen vendor's SDK to **create a session bound to a

persistent profile/context **(Browserbase Context `persist:true` / Steel `profileId` + `persistProfile:true` / Anchor profile / Hyperbrowser profile), navigates it to the platform `login_url` from `config/posting-platforms.json`, and returns the** live-view URL** (+ session/profile IDs).

1. The panel **embeds the live-view URL in an `<iframe>`** (interactive mode:

Steel `?interactive=true&showControls=true`; Browserbase `debuggerFullscreenUrl` without `pointer-events:none`; Anchor `live_view_url`; Hyperbrowser `liveUrl`). The operator logs in to Landmodo inside it — the app never sees the password.

1. **Detect success by polling**, not a vendor event (none of the four expose a

reliable "logged in" hook): the connect route or a companion `GET` checks the session for a post-login signal — a Landmodo auth cookie, or a redirect off `/login` to the seller dashboard URL.

1. **Persist**: on success, either store the profile/context ID for the poster to

`connectOverCDP` at run time, **or** export cookies via CDP `Network.getAllCookies` and write the existing `workers/posting/auth/<platform>.json` (option B keeps `post.ts` and the whole poster pipeline unchanged — the recommended low-risk path).

Files that would change (for the follow-up implementation PRD, not now):

- `dashboard/components/PostingAuthPanel.tsx` — add Connect/Re-connect button, the

live-view iframe, and login-success polling (currently a read-only status panel).

- `dashboard/app/api/posting-auth/route.ts` — today serves file-exists + mtime;

gains (or spawns a sibling route) the **create-session / connect** action and a **login-status poll**.

- **New** `dashboard/app/api/posting-auth/connect/route.ts` (or similar) — vendor

SDK session creation + live-view URL issuance.

- `config/posting-platforms.json` — per-platform capture config (e.g. a

`capture: "live-view"` flag and the post-login success signal to poll for).

- `workers/posting/post.ts` — unchanged under option B (cookie export); under

option A it swaps `storageState` load for `connectOverCDP` against the profile.

- `workers/posting/capture-login.ts` — superseded for enabled platforms; kept as a

local break-glass path.

- **New dependency**: the vendor SDK in `dashboard/package.json` (and/or

`workers/posting/`), plus a vendor **API key** stored as a secret in `.env.local` (same handling class as `workers/posting/auth/*.json`).

No vendor accounts were created and nothing was purchased for this evaluation — all pricing and capability data is from public pricing/docs pages, each cited with its fetch date above.

Sources (all fetched 2026-07-10): Browserbase pricing `https://www.browserbase.com/pricing`, live view `https://docs.browserbase.com/features/session-live-view`, contexts `https://docs.browserbase.com/features/contexts`; Steel pricing `https://steel.dev/pricing`, human-in-the-loop `https://docs.steel.dev/overview/sessions-api/human-in-the-loop`, profiles `https://steel.dev/blog/profiles`; Anchor pricing `https://docs.anchorbrowser.io/pricing`, live view `https://docs.anchorbrowser.io/advanced/browser-live-view`; Hyperbrowser live view `https://hyperbrowser.ai/docs/sessions/live-view`, pricing (JS-rendered; via search of `https://hyperbrowser.ai/docs/pricing`).

---

5. Other Options

Four alternatives to the hosted live-view path (§4) round out the field. Each is scored on the same rubric — **UX flow, build effort, reliability/fragility, security posture, cost** — and judged against the decided constraints: **single user now, the user always logs in themselves, and the app never stores**

marketplace passwords. Each ends with an explicit verdict line. None of these was built or purchased; this is a paper evaluation grounded in the seeded research plus the first-hand Landmodo recon in §3 and the live Apify pricing check below.

(a) Browser-extension cookie export (PhantomBuster-style)

A small browser extension reads the session cookies from the user's own logged-in browser and hands them to the app, which converts them into a Playwright `storageState` JSON. PhantomBuster is the canonical productized version (its extension auto-detects the LinkedIn `li_at` cookie + user agent and fills the session field on a "Connect" click); Apify documents the manual equivalent (log in normally, export cookies with an EditThisCookie-class extension as a JSON array, paste into the actor's *Initial cookies* input).

- **UX flow:** install the extension once → log in to Landmodo normally in your own

browser → click **Connect** in the dashboard → the extension POSTs the landmodo.com cookies (and, via a content script, any localStorage the auth needs) to a dashboard endpoint, which writes `workers/posting/auth/landmodo.json`. Good UX *after* install; the one-time install (and, for true external users, a Chrome Web Store listing or an "unpacked" sideload) is the friction.

- **Build effort:** moderate — **~2–4 days** for a minimal MV3 extension (`cookies`

permission scoped to landmodo.com + land.com, one popup, one authenticated fetch to the endpoint) plus the receiving route and the cookie-array → storageState conversion (trivial). Publishing to the Chrome Web Store adds review latency; sideloading unpacked is fine for one operator but ugly to distribute.

- **Reliability/fragility:** the weak spot. The session is minted on the **user's**

browser fingerprint and IP; replaying it from the worker's datacenter IP changes both, which is exactly the signal Landmodo's invisible reCAPTCHA (§3) and any future device-binding would score against. It works today for a small site but is the mismatch most likely to shorten session life or trip a silent re-auth. `HttpOnly` cookies are readable via the `chrome.cookies` API (no gap there); localStorage-based auth needs a content script. Refresh story is "click Connect again," which the extension keeps cheap.

- **Security posture:** cookies transit our API, so we hold session material in

flight and at rest — better than passwords (revocable, no credential-reuse blast radius) but worse than the live-view path where state never leaves the vendor. Mitigate with TLS, short-lived signed upload tokens, and encryption at rest. **Honors the no-password-custody constraint** (the app never sees the password).

- **Cost: ~$0** (a one-time $5 Chrome developer account if published).
- **Verdict: viable fallback / break-glass.** Constraint-compliant and cheap, but

the browser-extension install is friction the live-view path doesn't have, and the user-IP-to-server-IP replay mismatch makes its sessions more fragile than a vendor profile minted and replayed on the same IP class. Keep it as a no-vendor-dependency backup, not the primary.

(b) Apify (managed actors + session store)

Apify is a scraping/automation cloud. Relevant pieces: its **"Log in by transferring cookies" tutorial** (the same extension/manual-export pattern as option (a) — Apify ships no first-party capture extension, it

recommends EditThisCookie-class tools); community actors `pocesar/login-session` (logs in with username/password *inside* an actor and stores a named session — that's the option (d) credential-vault pattern, not capture), `Cookie & Session Manager`, and `Session/Login Extractor`; **SessionPool** (Crawlee) for per-session cookie/proxy/fingerprint persistence and rotation; and the option to run **our Playwright poster as an actor** (containerize `post.ts`, invoke via API with `{ listing, storageState }`).

- **UX flow:** for *capture*, identical to option (a) — extension or manual cookie

paste. Apify does not improve the login-capture step; it replaces the **worker infrastructure**, not the capture problem.

- **Build effort:** small port to run the poster as an actor (a Dockerfile + input

schema), but this **adds a vendor to solve a problem we don't have** — a working self-hosted poster already exists (`workers/run-poster.sh` + `workers/posting/`).

- **Reliability/fragility:** same cookie-replay fragility as (a) for capture;

Apify's managed residential proxies and Crawlee fingerprinting can *mitigate* the IP mismatch if we route posts through them, which is a genuine (if paid) edge over a bare extension.

- **Security posture:** cookies/sessions now live in a third-party cloud in addition

to transiting our API — a strictly larger surface than option (a) for the same capture UX. Still **no password custody** (unless one uses the `login-session` actor, which is option (d) and rejected).

- **Cost (live-verified `apify.com/pricing`, fetched 2026-07-10): Free** $0/mo

($5 platform credit); **Starter $29/mo** ($29 credit); **Scale $199/mo** ($199 credit); **Business $999/mo**; Enterprise custom. Compute is **$0.20/CU** on Free/Starter (1 CU = 1 GB-RAM-hour), $0.16 on Scale, $0.13 on Business; residential proxies **$8/GB** (Free/Starter). A 3-minute Playwright post at 2–4 GB ≈ 0.1–0.2 CU ≈ **$0.02–0.04/post** — trivial. (Seeded figures confirmed still current: Starter $29, ~$0.20/CU, $8/GB.) So ~$29/mo + a few cents/post if a paid plan is needed for concurrency; the Free tier's $5 credit covers our 1-user volume outright.

- **Verdict: rejected for our scale (not on constraint grounds).** Apify solves

worker hosting and proxy rotation, which we already have in-house cheaply, and does **nothing** for the capture UX that (a) doesn't already do. It only becomes attractive if we later want managed compute + proxies + scheduling as a bundle — not our problem at 1–20 users posting a few listings each.

(c) Self-hosted streamed browser (neko / noVNC / CDP screencast)

Build the live-view experience ourselves instead of renting it: (a) **neko** (`m1k1o/neko`) — Dockerized Chromium + WebRTC, explicitly pitched as an open-source Hyperbeam alternative, iframe-embeddable, sub-300 ms latency; (b) **Xvfb + x11vnc + noVNC** wrapped around the existing headed Playwright script; or (c) raw CDP `Page.startScreencast` + synthesized input over a WebSocket (what the §4 vendors build internally).

- **UX flow:** can match the hosted live-view path (§4) exactly once built — the

operator logs in to Landmodo inside an embedded stream in the dashboard, never sees a terminal, never installs anything.

- **Build effort:** the catch, and it's large. neko yields a *viewable* browser in

~a day, but a production capture flow still needs per-user container orchestration, profile persistence + extraction to `storageState`, session lifecycle, a TURN server for WebRTC through NATs, auth around the stream, and TLS — realistically **1–2 weeks to solid**, plus ongoing ops. CDP screencast from scratch is **2–4 weeks**. This is 5–10× the ~1–2 days of a hosted vendor for the same result.

- **Reliability/fragility:** sessions originate from our **own server IP** and stay

consistent thereafter (good for longevity — better than the extension's user→server mismatch), but there is **no vendor stealth or CAPTCHA assistance** — against Landmodo's invisible reCAPTCHA (§3) a human-driven login still passes fine, but we own every Chromium/driver update and every breakage.

- **Security posture:** best data sovereignty — nothing leaves our infrastructure,

no vendor to assess. The flip side is we own all of it, and a leaked neko/noVNC stream URL grants full browser control, so the stream needs the same short-lived, authenticated guarding the vendors provide out of the box. **No password custody** — the user logs in themselves.

- **Cost:** a **$10–40/mo VPS** plus our time (the dominant cost).
- **Verdict: viable fallback, deferred.** Constraint-compliant and the only option

with zero vendor dependency, but the effort and ops burden are unjustified at this scale when a hosted vendor gives the identical UX in ~1–2 days for $0–20/mo. Reach for it only if vendor dependency ever becomes a hard constraint (e.g. a compliance requirement that state never leave our infra).

(d) Credential vault (store username/password, headless login)

The user types their Landmodo email + password into an app form once; the app stores them encrypted and **logs in headlessly on their behalf** whenever a session is needed. This is what the `pocesar/login-session` Apify actor automates.

- **UX flow:** simplest possible — one form, and **self-healing**: when a session

expires the worker silently re-logs-in with no user involvement. On paper the best UX of any option.

- **Build effort:** low — a login script plus encrypted-at-rest secret storage

(libsodium sealed box / KMS). **~1–2 days.**

- **Reliability/fragility:** the worst fit for Landmodo specifically. §3 confirmed

the login is guarded by an **invisible Google reCAPTCHA** that scores requests in the background — a headless, datacenter-IP scripted login is the textbook trigger for exactly that defense, so this is the option most likely to **silently start failing at any time**. It also breaks outright if Landmodo ever adds MFA (the recon found none today, but a scripted flow can't answer an emailed/SMS code without extra plumbing). Automated login with stored credentials is also the pattern marketplace ToS most explicitly prohibit.

- **Security posture:** the disqualifier. The app would hold **reusable,

password-equivalent credentials for accounts it does not own**. Users reuse passwords across sites, so a breach of the vault is a breach of their accounts everywhere — a liability and a trust ask ("give this app your password") that every other option avoids.

- **Cost: ~$0.**
- **Verdict: rejected — violates the no-password-custody constraint.** The operator

decided the app never sees or stores marketplace passwords; a credential vault is *defined* by storing them, so it is out regardless of its UX appeal. Documented here only for completeness. (Even setting the constraint aside, §3's invisible reCAPTCHA would make headless login the most fragile choice on the board.)

Summary of §5 verdicts

Option	Constraint-compliant?	Verdict
(a) Extension cookie export	Yes (no password custody)	**Viable fallback / break-glass** — cheap and compliant, but install friction + user-IP→server-IP replay make sessions more fragile than a vendor profile.
(b) Apify	Yes	**Rejected for scale** — solves worker hosting we already have; no better capture UX than (a).
(c) Self-hosted neko/noVNC	Yes	**Viable fallback, deferred** — identical UX to §4 but 1–2 weeks + ops vs. 1–2 days; only if vendor dependency becomes unacceptable.
(d) Credential vault	**No** — stores passwords	**Rejected** on the no-password-custody constraint (and would be the most fragile against Landmodo's invisible reCAPTCHA anyway).

Sources (all fetched 2026-07-10): live Apify pricing `https://apify.com/pricing` (Starter $29/mo, $0.20/CU, $8/GB residential — seeded figures confirmed current); Apify cookie-transfer tutorial `https://docs.apify.com/platform/tutorials/log-in-by-transferring-cookies`; Apify session management `https://docs.apify.com/sdk/js/docs/guides/session-management`; `pocesar/apify-login-session` `https://github.com/pocesar/apify-login-session`; PhantomBuster extension onboarding `https://support.phantombuster.com/hc/en-us/articles/24572365472786`; neko `https://github.com/m1k1o/neko`. Landmodo reCAPTCHA / MFA findings are first-hand from §3 (`workers/posting/recon-landmodo.ts`, 2026-07-10).

---

6. Comparison & Recommendation

This section synthesizes §2–§5 into a single decision. The bottom line: **Land.com should move to the LandFeed XML API (§2) once support confirms eligibility, and Landmodo should use a hosted live-view browser — Browserbase (§4) — so the operator logs in inside the dashboard.** The rest of this section is the evidence for that call.

Comparison matrix

Every evaluated option scored on the shared rubric. UX / effort / reliability / security are qualitative (▲ good · ● middling · ▼ poor for our constraints); costs are monthly at 1 user (~10 posts/mo) and at 20 users (~30 posts/mo each), from the §4 cost table and the §5 per-option costs. The current CLI flow (§1) is the baseline every alternative is judged against.

Option (§)	UX	Effort	Reliability	Security	$/mo now	$/mo @ 20 users
Current CLI capture (§1, baseline)	▼ terminal on the worker box; read-only dashboard	— (exists)	● session rots silently; first signal is a failed post	▲ no password custody; `600` local files	$0	$0 (but doesn't scale to non-technical users)
LandFeed XML API — land_com (§2)	▲ no login at all; feed replaces capture	● build XML feed + photo hosting; **gated on eligibility**	▲ no session to expire; official sanctioned path	▲ shared key, no cookies, no passwords	$0	$0
Browserbase — live-view (§4)	▲ log in inside the dashboard iframe	● ~1–2 days SDK + iframe + poll	▲ same-IP-class replay; **CAPTCHA solving bundled**	▲ state stays vendor-side; no password custody	$0 (Free, 15-min cap)	**$20** (Developer)
Steel.dev — live-view (§4)	▲ same as Browserbase	● ~1–2 days	▲ same-IP replay; CAPTCHA solving	▲ vendor-side; no passwords	$0 (one-time $30 credits)	**$250** (no cheap tier) or self-host
Anchor — live-view (§4)	▲ same; one-time hand-off URL	● ~1–2 days	▲ same-IP replay	▲ vendor-side; no passwords	$0 ($5/mo credits)	~$8 PAYG (or $50 floor)
Hyperbrowser — live-view (§4)	▲ same as Browserbase	● ~1–2 days	▲ same-IP replay	▲ vendor-side; no passwords	$0 (1,000 credits)	~$30 (Startup)
Extension cookie export (§5a)	● good after a one-time install	● ~2–4 days (MV3 ext + route)	▼ user-IP→server-IP replay mismatch shortens sessions	● cookies transit our API	~$0	~$0
Apify (§5b)	● capture no better than (a)	● small actor port, but adds a vendor	● (a)'s fragility, mitigable via paid proxies	● cookies also live in a 3rd-party cloud	$0 (Free $5 cr)	~$29 + cents/post
Self-hosted neko/noVNC (§5c)	▲ matches §4 once built	▼ 1–2 weeks + ongoing ops	● same-server IP, but we own every breakage	▲ nothing leaves our infra	~$10–40 VPS	~$10–40 VPS
Credential vault (§5d)	▲ best on paper (self-healing)	▲ ~1–2 days	▼ headless login trips Landmodo's invisible reCAPTCHA	▼ **stores passwords for accounts we don't own**	~$0	~$0

Reading the matrix: the LandFeed API dominates for land_com (it removes the problem rather than improving it), and among the browser-session options for Landmodo the four hosted live-view vendors are near-identical on everything except price at scale, where **Browserbase's $20/mo (with CAPTCHA solving bundled, which directly addresses Landmodo's invisible reCAPTCHA from §3) is the cheapest turnkey recurring option**. The credential vault is the only option that fails a hard constraint (password custody, ▼ security) and is separately the most fragile against Landmodo's reCAPTCHA; extension export and self-hosting are constraint-compliant but each trade away either reliability or effort for no offsetting gain over a hosted vendor.

Per-platform recommendation

Land.com → LandFeed XML API (§2). Fallback: hosted live-view capture (the Landmodo path below), keeping the current Playwright poster. The LandFeed feed is Land.com's own sanctioned integration: one authenticated HTTPS POST adds/updates/deletes listings with a shared key that never expires and no browser session to capture, at $0 beyond the existing plan. It eliminates the Land.com auth-capture problem outright rather than merely improving its UX. The one open item is eligibility — the spec names a Corporate Account as a prerequisite, so the recommendation is contingent on the support answer to the §2 draft email; if feed access is denied, Land.com falls back to the same hosted live-view capture we recommend for Landmodo, with zero change to the existing poster mechanics.

Landmodo → hosted live-view browser, Browserbase (§4). Fallback: browser-extension cookie export (§5a), then self-hosted neko (§5c). Landmodo has no API (§3) and its login is a plain email/password form guarded only by an invisible reCAPTCHA, so the right fit is a human logging in inside a live-view iframe in our own dashboard — the person passes reCAPTCHA naturally and we persist the resulting session. Browserbase is the pick: battle-tested, $0 at our current scale and $20/mo at 20 users, with CAPTCHA solving bundled, and it feeds state back to the poster via a documented CDP cookie export that leaves `post.ts` unchanged. The named fallback if we ever want zero vendor dependency is the extension cookie export (cheap, constraint-compliant, more fragile), and beyond that self-hosted neko for full data sovereignty at the cost of 1–2 weeks of build and ops.

What would change this decision

- **LandFeed eligibility denied.** If Land.com support says the feed requires a

Corporate Account tier we can't or won't reach, land_com drops to the hosted live-view fallback and the LandFeed recommendation is shelved (revisit if we later upgrade the account).

- **LandFeed photo hosting proves impractical.** The feed ingests photos by public

URL (§2); if we can't serve `outputs/<taskId>/photos/` at a public HTTPS URL acceptably, that raises the feed's effort enough to reconsider live-view for land_com too.

- **Landmodo adds real bot protection or MFA.** §3 found only an invisible reCAPTCHA

and no MFA. If Landmodo later adds a visible challenge, device binding, or MFA, the extension and credential-vault paths degrade further and the hosted live-view vendors' CAPTCHA solving / human-in-the-loop becomes more valuable, not less — reinforcing the recommendation rather than overturning it.

- **Vendor pricing drift.** Pricing moved once already during this spike

(Steel.dev's cheap tier vanished, §4). If Browserbase's $20 Developer tier changes or its Free tier's 15-min session cap starts truncating logins, re-rank against Anchor (~$8 PAYG) and Hyperbrowser (~$30) using the §4 cost table before committing.

- **Scale or compliance shift.** If usage grows past ~20 users, or a compliance rule

requires that session state never leave our infrastructure, self-hosted neko (§5c) moves from deferred fallback to primary.

- **Landmodo ships an API.** If the support email (§3 operator task) reveals a

bulk-import or feed option, Landmodo could follow the same API-first path as land_com and skip browser capture entirely.

UX and reliability gains over the current CLI flow

Every recommended path is a strict improvement over the terminal ritual in §1. Concretely, the operator gains:

- **No terminal.** Capture becomes an in-app **"Connect"** button in

`PostingAuthPanel.tsx` (or vanishes entirely for land_com under LandFeed) instead of `npm run capture-login -- <platform>` on the command line.

- **Capture from any device.** The live-view flow runs through the dashboard, which

is already reachable via the Cloudflare tunnel — an expired session can be fixed from a phone or laptop away from the worker machine, which §1's headed-Chromium + stdin ritual made impossible.

- **In-app re-auth when a session expires.** The dashboard gains a real

"Re-connect" action and a login-success check, replacing today's read-only file-exists/mtime panel.

- **No silent expiry surprise.** §1's first signal of a dead session is a failed

posting discovered after approval. A live-view Connect flow lets the operator refresh proactively; and for land_com, LandFeed removes the expiring session altogether.

- **Usable by non-technical users.** No npm, no repo layout, no terminal knowledge —

a second operator or assistant can self-serve a login refresh, lifting the §1 cap of "one technical operator."

- **Longer-lived sessions via same-IP replay.** A vendor profile is minted and

replayed from the same IP class (and, with Browserbase, backed by CAPTCHA solving), so sessions survive longer than the §5a extension's user-IP→server-IP mismatch — and LandFeed's shared key never expires at all.

- **No two-context juggling.** The browser-window-plus-terminal-Enter dance of §1

collapses to a single in-dashboard interaction.

- **Preserves what already works.** No password custody, secrets kept out of git and

the file API, and `600`-locked local state all carry forward — the recommended paths improve the UX without giving up §1's security properties.

7. Proposed Implementation PRD

*This is a ready-to-run PRD for the approaches recommended in §6. If accepted as-is, it can be saved to `specs/auth-capture.md` with `**Status:** Draft` and handed to Ralph. It carries forward the two per-platform decisions — **Land.com → LandFeed XML API** and **Landmodo → Browserbase hosted live-view capture** — and sequences them config/schema → backend → UI. Vendor and marketplace onboarding are the operator's to do first; they are flagged as **Operator Tasks** below, not Ralph stories.*

Introduction

Replace the terminal-only login-capture ritual (§1) with two in-app, constraint-compliant paths: post Land.com listings through its sanctioned **LandFeed XML API** (no browser session at all), and capture the **Landmodo** login inside the dashboard via a **Browserbase** hosted live-view iframe, persisting the resulting session back into the poster's existing `auth/landmodo.json` storageState. The app never sees or stores a marketplace password in either path.

Goals

- Eliminate the CLI capture step for both enabled platforms: Land.com posts via feed,

Landmodo re-auth is an in-dashboard **Connect** button reachable from any device.

- Keep the existing poster pipeline (`workers/run-poster.sh` → `post.ts`) working with

the smallest possible change — the Landmodo path exports cookies into the same `auth/<platform>.json` file the poster already loads (§4 "low-risk path").

- Preserve every §1 security property: no password custody, secrets gitignored and

`600` , never logged, never in the file API.

- Make session expiry recoverable proactively (a real re-auth action) instead of

discovered by a failed post.

Non-Goals

- **No multi-user identity/accounts** — single operator, as decided; the design may be

multi-user-friendly but building it out is out of scope.

- **No credential vault / stored passwords** (§5d, rejected) — the user always logs in

themselves.

- **No change to the other four platforms** (land_century, landflip, land_listings,

landhub) — they stay on manual/CLI capture.

- **No ad-copy/generation changes** — this is purely auth capture + Land.com transport;

char caps and DREAMS stay where they are (`config/ad-platforms.json` , generate-ad).

Operator Tasks (prerequisites — NOT Ralph stories)

These are human steps that must happen outside the code and gate specific stories. Ralph cannot do them; each blocked story notes the gate.

- **OT-1 — Obtain LandFeed access.** Send the §2 draft email to Land.com support;

confirm LandFeed eligibility on the account (Corporate Account if required), obtain the `loa_shared_key` , and confirm the `loa_account_id` . **Gates US-104's live submission** (the feed builder and validation can be built and tested against the XSD in `--test` /dry mode without it).

- **OT-2 — Create a Browserbase account + API key.** Sign up, create an API key and a

reusable **Context** for Landmodo, and put the key in `.env.local` (same secret class as `auth/*.json`). **Gates US-202/US-203 live capture** (the route and UI can be built and typechecked with the key absent, failing closed with a clear message).

- **OT-3 — Decide public photo hosting for LandFeed.** LandFeed ingests photos by

public HTTPS URL (§2); pick where `outputs/<taskId>/photos/` images are served (e.g. an R2/S3 bucket or the Cloudflare tunnel). **Informs US-103.**

Land.com — LandFeed XML API stories

US-101: LandFeed config + secret plumbing

Description: Add a `landfeed` config block to `config/posting-platforms.json` (feed endpoint `https://www.land.com/LandFeed/` , `mode: "test" | "live"` , and the non-secret `loa_account_id`), and read `LOA_SHARED_KEY` from the environment — never committed. A tiny loader validates the block and fails closed with a clear message when the key is absent.

Acceptance Criteria:

- [ ] `config/posting-platforms.json` `land_com` entry gains a `landfeed` object:

`{ endpoint, mode, account_id, account_email }` ; `mode` defaults to `"test"`

- [ ] A loader (e.g. `workers/posting/landfeed/config.ts`) reads the block + `LOA_SHARED_KEY` from env, throws a one-line "set LOA_SHARED_KEY" error when missing (mirrors the poster's "run capture-login" fail-fast)
- [ ] The shared key is never written to config, logs, or the file API; `.env.local` handling matches existing secrets
- [ ] Typecheck passes (`cd workers/posting && npm run typecheck`)

US-102: Build a schema-valid LandFeed XML document from tasks

Description: Write a builder that turns the operator's active Land.com inventory (approved ad tasks) into one LandFeed XML document — `<channel>` credentials plus one `<item>` per listing — mapping task/metadata

fields to the §2 schema (id, description CDATA, listing_title, county/state/closest_city, lot_size, price, propertytypes bitmask, listing_status). It validates against the live XSD and refuses HTML/URLs/ emails in `description`.

Acceptance Criteria:

- [ ] `workers/posting/landfeed/build-feed.ts` maps a list of tasks → a LandFeed XML string, credentials pulled from US-101's loader
- [ ] Output validates against `https://www.land.com/LandFeed/schemas/LandFeedSchema1.0.xsd` (fetch once, validate locally, e.g. with `libxmljs` / `fast-xml-parser` + assertions); a unit check with one sample task passes
- [ ] `description` is CDATA-wrapped and rejects HTML tags, URLs, and email addresses (per §2); `propertytypes` is the bitwise sum, max 3 types
- [ ] **Feed-is-authoritative guard:** the builder emits the operator's *entire* active inventory (a comment/asserted invariant documents that an omitted id is a DELETE)
- [ ] Typecheck passes

US-103: Public photo URLs for feed items

Description: LandFeed pulls photos by URL, but the app stores them locally at `outputs/<taskId>/photos/`. Add a resolver that maps each local photo to a public HTTPS URL (per OT-3's chosen host) and emits `image_link` + `loa_photo_tour_images` for each item, honoring the ≤2 MB / min-width rules.

Acceptance Criteria:

- [ ] A resolver (`workers/posting/landfeed/photo-urls.ts`) maps `outputs/<taskId>/photos/*` → public URLs using a configured base from OT-3; primary photo → `item.image_link`, rest → `loa_photo_tour_images.image_link` (0–200)
- [ ] Photos exceeding 2 MB or below the min width are skipped with a logged warning rather than emitted (feed would reject them)
- [ ] The base URL is config-driven (no hardcoded host), absent-config fails closed with a clear message
- [ ] Typecheck passes

US-104: Submit the feed (test mode first) and record results

Description: POST the built XML to the LandFeed endpoint over HTTPS with the §2 transport headers (`Content-Type: text/xml`, TLS 1.2+, `User-Agent`, `Content-Length`), defaulting to `mode: test` so nothing goes live until the operator flips it. Parse the response, and record per-task feed status on the task (reusing the `postings[]` shape).

Acceptance Criteria:

- [ ] `workers/posting/landfeed/submit.ts` POSTs the US-102 document with the correct headers; `mode` comes from config (default `test`)
- [ ] In `test` mode it runs full validation without touching live listings (per §2) and logs the returned validation/warning/error report

- [ ] Each task's `land_com` posting entry is PATCHed with the feed outcome (`posted` / `failed` + `lastError`) via the API, never by direct file write (matches the §"Ad posting" invariant)
- [ ] **Live submission is gated on OT-1** (shared key); with the key absent it fails fast per US-101 and does not attempt a POST
- [ ] Typecheck passes

Landmodo — Browserbase live-view capture stories

US-201: Live-view capture config

Description: Add per-platform capture config to `config/posting-platforms.json` so the dashboard knows which platforms use hosted live-view and how to detect a successful login: a `capture` flag (`"live-view" | "cli"`) and a `login_success` signal (an auth cookie name and/or a post-login URL to detect a redirect off `/login`). Read the Browserbase API key from env.

Acceptance Criteria:

- [ ] `landmodo` entry gains `capture: "live-view"` and a `login_success` object (`{ cookie?, redirect_off?: "/login" }`); other platforms default to `"cli"`
- [ ] `BROWSERBASE_API_KEY` (and the Landmodo Context ID) are read from env; absent key → clear fail-closed error (no crash)
- [ ] `GET /api/posting-platforms` (existing) serves the new fields so the client can branch on `capture` without importing config directly (per AGENTS.md)
- [ ] Typecheck passes (`cd dashboard && npx tsc --noEmit`)

US-202: Connect + login-status API routes

Description: Add `POST /api/posting-auth/connect` that creates a Browserbase session bound to the persistent Landmodo **Context** (`persist:true`), navigates it to Landmodo's `login_url` , and returns the live-view URL + session/context IDs; plus a `GET /api/posting-auth/status` that polls the session for the US-201 `login_success` signal. On success it exports cookies via CDP `Network.getAllCookies` and writes `workers/posting/auth/landmodo.json` — leaving `post.ts` unchanged.

Acceptance Criteria:

- [ ] `dashboard/app/api/posting-auth/connect/route.ts` creates a persistent-context Browserbase session, navigates to the `login_url` from config, returns `{ liveViewUrl, sessionId }`
- [ ] `dashboard/app/api/posting-auth/status/route.ts` (or a `?poll=` action) checks the session for the configured post-login signal and reports `{ loggedIn: boolean }`
- [ ] On detected login it writes `auth/landmodo.json` in Playwright storageState shape via CDP cookie export; the file is `600` and gitignored (matches existing secret handling); **poster and `post.ts` are unchanged**
- [ ] Both routes fail closed with a clear message when `BROWSERBASE_API_KEY` is absent (OT-2 gate); no secret is logged or returned to the client
- [ ] Typecheck passes

US-203: "Connect" UI in PostingAuthPanel

Description: Turn the read-only `PostingAuthPanel.tsx` (§1) into an actionable panel: for `capture: "live-view"` platforms, render a **Connect / Re-connect** button that calls US-202, embeds the returned live-view URL in an interactive `<iframe>`, and polls `status` until login is detected — then closes the iframe and shows the saved session (existing mtime display). CLI platforms keep today's read-only status.

Acceptance Criteria:

- [ ] `PostingAuthPanel.tsx` shows a Connect (or Re-connect, when a session exists) button only for `capture: "live-view"` platforms; CLI platforms are unchanged
- [ ] Clicking Connect embeds the live-view URL in a sandboxed interactive `<iframe>` and polls `GET /api/posting-auth/status`; on `loggedIn: true` it tears down the iframe and refreshes the saved-session display
- [ ] Error and timeout states are surfaced in-panel (session create failed / key missing / login not detected within N minutes), matching the app's existing status-chip styling — no new modal (per §6 UX rules)
- [ ] **Browser verification:** with the dev server on port 3001 (per the Ad-photos verification gotcha), load `/settings`, confirm the Connect button renders for Landmodo and the iframe mounts on click (mock the connect route if OT-2's key is absent)
- [ ] Typecheck passes (`cd dashboard && npx tsc --noEmit`)

Story sequencing

Dependency order is **config/schema → backend → UI**, per platform, and the two platforms are independent (either can ship first):

- **Land.com:** OT-1 → US-101 → US-102 → US-103 → US-104 (US-104 live submit gated on OT-1).
- **Landmodo:** OT-2 → US-201 → US-202 → US-203 (US-202/203 live capture gated on OT-2).

Every Ralph story above is describable in 2–3 sentences, ends in "Typecheck passes," and each UI story (US-203) additionally requires browser verification. The two operator prerequisites (OT-1, OT-2, plus the OT-3 hosting decision) are called out separately and are not Ralph work.